# Implementation Review of Revision 43

Neighbor-Layer Hypergraph Contrastive Learning ⟶ MMHCL+

**Executive Summary**

I carefully read the entire TEX specification file (revision 43) and cross-checked every component against the Python source code. The system implements most of the design correctly; however, **five important deviations** from the TEX description require attention. Overall implementation completeness is estimated at $\approx$ **75%**. The training loop, loss functions, and scaffold structure are already solid. The three core novelty elements — EMA teacher, real GradNorm, and FAISS-based Soft Topology-Aware Purification — still need to be added to reach 100%.

## 1. Components Implemented Correctly at 100%

| # | Component (TEX Section) | Implementation File | Status |
|---|---|---|---|
| 1 | Expanded Projector $d = 64 \to 2048 \to 8192$, 3-layer MLP + BN + ReLU (§4.4) | `projector.py`, `config.py` | ✔ Exact |
| 2 | Barlow Twins loss + optional `soft_weights` (§4.4) | `losses.py` — `barlow_twins_loss()` | ✔ Exact |
| 3 | Chunked InfoNCE + `dynamic_weights` + `chunk_size` (§4.4) | `losses.py` — `chunked_info_nce_loss()` | ✔ Exact |
| 4 | Soft BYOL alignment + stop-gradient (§4.4 Algorithm 1, Step 6) | `soft_byol.py` — `.detach()` on target | ✔ Exact |
| 5 | Neighbor-layer pairs $(E^{(l)}, E^{(l+1)})$ + stop-grad (§4.3) | `neighbor_pairs.py` | ✔ Exact |
| 6 | Dirichlet Energy $\mathrm{tr}\left(E^\top (I - \Theta) E\right)/n$ (§3.3) | `dirichlet.py` + inline `sparse_dirichlet_energy()` | ✔ Both correct |
| 7 | Temperature annealing $\tau = \max(\tau_{\min}, \tau_{\max} \cdot \gamma^t)$ (§4.4) | epoch loop in `main_mmhcl_plus.py` | ✔ Exact |
| 8 | Warmup epochs — BPR-only phase (§4.4) | `main_mmhcl_plus.py` — `in_warmup` gate | ✔ Exact |

| # | Component (TEX Section) | Implementation File | Status |
|---|---|---|---|
| 9 | 5-task loss aggregation (BPR, u2u, i2i, align, Dir) (Algorithm 1, Step 9) | `gradnorm.py` + `balancer.combine()` | ✔ All 5 present |
| 10 | `WEMAManager` — lazy row fetch + periodic update (§4.2) | `dynamic_ema_weights.py` | ✔ Correct |
| 11 | `MMHCLWithLayers` — layer caching for ii and uu branches | `main_mmhcl_plus.py` — `forward_plus()` | ✔ Correct caching |
| 12 | Package-level extension, original MMHCL backbone preserved intact (§4.1) | `mmhcl_plus_scaffold/` | ✔ Correct |

The components above map one-to-one to Algorithm 1, Listings 1–2, and Figures 3–4 in the TEX specification.

## 2. Five Critical Deviations That Must Be Fixed

### 2.1. Deviation 1 — Missing EMA Teacher Network (Algorithm 1, Steps 6 and 11)

**Critical — EMA teacher network is completely absent**

The target view is currently taken directly from the online bipartite embeddings, which **violates the core BYOL design principle** described in the TEX specification.

The TEX document states this clearly: Soft BYOL alignment must use an EMA teacher network $f_\xi$ — a momentum-updated copy of the encoder — to generate the target view, followed by applying $sg(\cdot)$ (stop-gradient). This is the core idea inherited from BYOL (Grill et al., 2020). Figure 4 in the TEX specification explicitly shows an *EMA Teacher Network (momentum = 0.99)* as a separate processing stream.

**Current code behavior:** `main_mmhcl_plus.py` directly uses `i_bip` / `u_bip` (bipartite embeddings) with `.detach()`:

Listing 1: Current (incorrect) alignment call

```
batch_aln = (
    soft_byol_alignment(ii_final, i_bip) # missing EMA teacher!
    + soft_byol_alignment(uu_final, u_bip)
)
```

**Required fix:** Add an EMA teacher model as follows.

- `ema_model = copy.deepcopy(model)`
- a decorated function `@torch.no_grad() update_ema(student, teacher, momentum=0.99)`
- invoke the EMA update at the end of every mini-batch (Algorithm 1, Step 11):

$$\xi \leftarrow \beta\,\xi + (1-\beta)\,\theta$$

The target view must then be produced from `ema_model.forward_plus(...)` rather than from the online bipartite embeddings directly.

### 2.2. Deviation 2 — "GradNorm" Is Only a Learned Softmax, Not Real GradNorm

**Warning — Completeness ≈ 30%; only the weighting structure exists**

`self.alpha` is stored but never participates in any gradient-norm computation. The module behaves as a simple learnable weighting layer.

TEX §4.4 explicitly refers to a *GradNorm Dynamic Balancing System*. The actual GradNorm algorithm (Chen et al., 2018) requires:

1. computing the gradient norm of each task loss with respect to shared parameters,
2. computing the relative training rate of each task,
3. performing a secondary update step on task weights to equalize those rates.

**Current code in `gradnorm.py`:**

Listing 2: Current `combine()` – only softmax weighting, no gradient norms

```python
def combine(self, losses):
    weights = self.normalized_weights() # only softmax(logits)
    return (weights * loss_vec).sum() # alpha is never used
```

`self.alpha` is stored but never participates in the computation. Therefore this module is effectively a *learnable weighting layer*, not true GradNorm. The `combine()` function must be rewritten to implement genuine gradient-norm equalization.

### 2.3. Deviation 3 — Missing FAISS/LSH and Soft Topology-Aware Purification (§3.4)

**Warning — Completeness ≈ 20%; this is the largest algorithmic gap**

Only dense cosine similarity is computed. The Jaccard term, bridge-score penalty, and FAISS/LSH backend are entirely absent.

TEX §3.4 describes the Soft Topology-Aware Purification mechanism in detail, using four signals:

- cosine similarity,
- sampled Jaccard coefficient,
- popularity penalty (bridge score),
- percentile score,

combined through the `soft_topology_weight()` formula and accelerated with ANN/FAISS so that complexity becomes $\mathcal{O}(N \cdot k)$ instead of $\mathcal{O}(N^2)$.

**Current code behavior:** `WEMAManager` uses only a dense cosine similarity matrix. There is no Jaccard term, no bridge-score penalty, and no FAISS/LSH backend. This is the largest deviation in terms of algorithmic novelty.

### 2.4. Deviation 4 — ASW Applied Only on the Item Side; User Side Missing

**Critical — `w_ema_u` for users does not exist**

`soft_weights` is not passed when Barlow Twins is called for the u2u branch, meaning the user-side branch is effectively unweighted.

TEX Listing 2 (§4.4) clearly applies Adaptive Sample Weighting (ASW) to **both** branches:

Listing 3: Expected ASW usage from TEX Listing 2

```
soft_weights = FAISS_EMA_W['u2u'][batch_users] # u2u branch
dynamic_weights = FAISS_EMA_W['i2i'][batch_items] # i2i branch
```

**Current code behavior:**

- only `self.w_ema_i` exists for items;
- `w_ema_u` for users is absent;
- `soft_weights` is not passed when Barlow Twins is called for the u2u branch.

Current (incorrect) call pattern:

Listing 4: Current (incorrect) u2u branch call

```
u2u_terms.append(
    barlow_twins_loss(z1, z2, lambd=args.barlow_lambda)
    # missing: soft_weights = w_ema_u.get_batch_weights(users)
)
```

### 2.5. Deviation 5 — Profiling-Guided Checkpointing Not Wired into `forward_plus()`

**Warning — Completeness ≈ 50%; module exists but is never invoked**

`checkpointing.py` correctly implements the intended logic, but it is not called anywhere inside `main_mmhcl_plus.py`.

TEX §4.3 Table 2 specifies: *"Checkpoint deep layers only; cache shallow layers ($g \leq 2$)."* The file `checkpointing.py` already implements this logic correctly.

**Current code behavior:** `MMHCLWithLayers.forward_plus()` calls `torch.sparse.mm()` directly without wrapping through checkpointing:

Listing 5: Current (uncheckpointed) forward pass

```
for _ in range(self._n_item_layers):
    ii_emb = torch.sparse.mm(I2I_mat, ii_emb) # not checkpointed!
```

The module `profiling_guided_checkpoint` exists in the scaffold but is not invoked anywhere inside `main_mmhcl_plus.py`.

## 3. Completion Summary

| Component | TEX Section | Completion | Notes |
|---|---|---|---|
| Package structure | §4.1 | ✔ 100% | Even better than the TEX scaffold |
| Expanded Projector | §4.4 | ✔ 100% | $d \to 2048 \to 8192$ is correct |
| Barlow Twins | §4.4 | ✔ 100% | Both code and math are correct |
| Chunked InfoNCE | §4.4 | ✔ 100% | Chunking + dynamic weights are correct |
| Neighbor-layer pairs | §4.3 | ✔ 95% | Missing explicit `max_hops` $= g$ limit |
| Temperature annealing | §4.4 | ✔ 100% | Formula is correct |
| Warmup epochs | §4.4 | ✔ 100% | BPR-only phase is correct |
| Dirichlet energy | §3.3 | ✔ 100% | Both implementations are correct |
| EMA Teacher Network | §4.4 Alg. 1 | ✘ 0% | Completely missing |
| GradNorm | §4.4 | **30%** | Only learned weighting, not real GradNorm |
| FAISS + Soft Purification | §3.4, §4.2 | **20%** | Only cosine similarity exists |
| ASW for u2u branch | §4.4 | ✘ 0% | Implemented only on the item side |
| Checkpoint in forward pass | §4.3 | **50%** | Module exists but is not wired in |

Overall, the system currently matches approximately **75%** of the TEX design. The training loop, loss functions, and scaffold structure are already solid. However, the three core elements that define the paper's novelty — the EMA teacher, real GradNorm, and FAISS-based Soft

Topology-Aware Purification — must still be added in order to reach revision 43 at 100%.